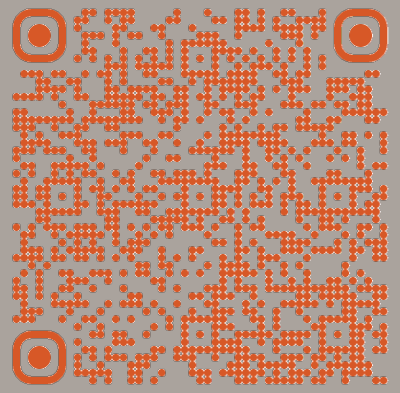
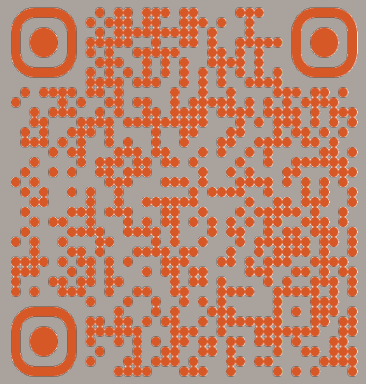
Python Foundations for Data Science

Week(2): Control Flow & Logic



GitHub Repository



Moodle

Dr. Ghadeer Mobasher

Agenda

1. **Logical Reasoning in Programming**

   Conditional statements, decision structures, and shorthand conditional (Ternary operator)

2. **Loops in Programming**

   For Loop, Nested Loops, Loop Control statements, and While Loop

3. **Basic List Comprehensions as Compact Logic Expressions**

   List Comprehension

4. **Putting it All Together**

   Mini Challenge

Logical Reasoning in Programming

Explanation:
Logical reasoning allows the program to make decisions based on given conditions. It is essential for dynamic behaviour in programs.

Example:

```
# Check user access based on role
user_role = "admin"

if user_role == "admin":
    print("Access granted to admin panel.")
else:
    print("Access restricted.")
```

Conditional Statements and Decision Structures

Conditional statements allow a program to execute different blocks of code depending on conditions

```python
# ------------------------------------------------------------
# if / elif / else — Age Classification
# ------------------------------------------------------------
# Python evaluates each condition from top to bottom and
# executes ONLY the first matching branch.
# 'elif' (else-if) lets us chain additional conditions
# without nesting. 'else' acts as the default fallback.
# ------------------------------------------------------------
age = 18

if age >= 18:          # Branch 1: adult (18 or older)
    print("You are an adult.")
elif age >= 13:        # Branch 2: teenager (13–17)
    print("You are a teenager.")
else:                  # Branch 3: child (below 13)
    print("You are a child.")
```

Python

```
You are an adult.
```

User Input and Role-Based Routing

```python
# ------------------------------------------------------------
# User Input + Role-Based Routing
# ------------------------------------------------------------
# input() reads a string typed by the user.
# .strip() removes any leading/trailing whitespace.
# .lower() normalises to lowercase so 'Admin' and 'ADMIN'
# are both treated the same as 'admin' (case-insensitive).
# ------------------------------------------------------------

# Ask the user to enter their role
role = input("Enter your role (admin/teacher/student): ").strip()

# Check the role and respond
if role.lower() == "admin":
    print("Access granted to admin panel.")
elif role.lower() == "teacher":
    print("Welcome, teacher")
elif role.lower() == "student":
    print("Welcome, student")
else:                                # role string not recognised
    print("Wrong input")
```

```
Wrong input
```

In **Week2.ipynb** , check the beginning of the notebook until the section “ **Shorthand Conditional (Ternary Operator) in Python”**

In **Week2_Exercise.ipynb** , you have 20 minutes to solve “Section A”.

First, download this Colab Notebook

Shorthand Conditional (Ternary Operator)

```
# ------------------------------------------------------
# Ternary (Inline) Conditional
# ------------------------------------------------------
# Python's ternary form collapses a two-branch if/else into
# a single expression:
#   <value_if_true> if <condition> else <value_if_false>
a = 10
result = "Positive" if a > 0 else "Not positive"
print(result)
```

Loops in Progamming

Loops in Programming

A **loop** is a control flow structure that repeatedly executes a block of code as long as a specified condition is true or for a predefined number of iterations. Loops are fundamental for automating repetitive tasks, reducing code duplication, and managing iterative computations.

For Loop

Example:

```
# For loop: iterates over a sequence (list, tuple, string, range)
for i in range(5):
    print(i)  # prints 0 to 4
```

Usage:

- Iterating through elements of a list
- Generating a fixed number of repetitions
- Creating new lists using list comprehensions

For Loop (Iterating over a List)

```python
# ------------------------------------------------------------
# for loop – Iterating Over a List
# ------------------------------------------------------------
# A 'for' loop visits each element of a sequence in order.
# 'number' is a loop variable – a temporary name that takes
# the value of each item one at a time. It only exists
# inside the loop body (the indented block below 'for').
# ------------------------------------------------------------
numbers = [0, 1, 2, 3, 4, 5]
for number in numbers:   # number is temporary name to refer to the list's items, valid only inside this loop
    print(number)
```

Nested Loops

Nested Loops

Usage:

- Iterating over multi-dimensional data (e.g., matrices, grids)
- Combining multiple sequences

```
# ------------------------------------------------------------
# Nested for loops — Grid / Coordinate Pairs
# ------------------------------------------------------------
# For each value of the OUTER loop variable 'i' (0, 1, 2),
# the INNER loop runs its FULL cycle through 'j' (0, 1).
# This produces every combination of (i, j) — essentially
# generating a 3×2 grid of coordinate pairs.
# Total iterations = len(range(3)) × len(range(2)) = 6
# ------------------------------------------------------------
for i in range(3):        # outer loop: 0, 1, 2
    for j in range(2):    # inner loop: 0, 1  (runs fully for each i)
        print(i, j)
```



```
0 0
0 1
1 0
1 1
2 0
2 1
```

Loop Control Statements: Break

break

- **Purpose:** Immediately exits the loop, even if the loop condition has not finished.
- **Use case:** Stop iterating when a specific condition is met.

```
# ------------------------------------------------------------
# break — Early Exit from a for Loop
# ------------------------------------------------------------
# 'break' immediately terminates the loop, jumping past
# the remaining iterations and any 'else' clause.
# Here we iterate over a tuple (immutable ordered sequence).
# When number == 3, we print it, then 'break' fires —
# so 4 and 5 are never reached.
# Output: 0, 1, 2, 3
# ------------------------------------------------------------
numbers = (0, 1, 2, 3, 4, 5)   # tuple — same syntax as list but immutable
for number in numbers:
    print(number)
    if number == 3:
        break                  # stop the loop immediately
```



```
0
1
2
3
```

Loop Control Statements: Continue

continue

- **Purpose:** Skips the current iteration and moves to the next iteration of the loop.
- **Use case:** Skip processing certain elements while continuing the loop.

```
# ------------------------------------------------------------
# continue — Skip Even Numbers, Print Only Odds
# ------------------------------------------------------------
# 'continue' skips the REST of the current loop body and
# jumps straight to the next iteration — the loop does NOT
# terminate (unlike 'break').
# When i % 2 == 0 (even), we skip the print statement.
# When i % 2 != 0 (odd), the if is False, so we reach print.
# range(1, 10) generates 1 through 9 (10 is excluded).
# ------------------------------------------------------------
for i in range(1, 10):
    if i % 2 == 0:
        continue            # skip even numbers — jump to next i
    print("Odd number:", i)
```



```
Odd number: 1
Odd number: 3
Odd number: 5
Odd number: 7
Odd number: 9
```

Generator Expression: Conditional Filtering

```
# ------------------------------------------------
# Generator Expression: Conditional Filtering
# ------------------------------------------------

# A list of student scores
scores = [85, 45, 72, 38, 90]

# Create a generator expression:
# - 'for score in scores' checks each score in the list.
# - 'if score >= 50' keeps only scores of 50 or higher.
# - The first 'score' tells Python what value to produce.
# - The generator produces matching scores one at a time,
#   instead of storing them all in a new list.
passed_scores = (score for score in scores if score >= 50)

# Print each passing score one at a time
for score in passed_scores:
    print(score)
```

```
# Output:
# 85
# 72
# 90
```

Tells Python which value to return.

While Loop

While Loop

Executes as long as a condition is true.

Example:

```
count = 0
while count < 5:
    print(count)
    count += 1
```



```
0
1
2
3
4
```

> **Usage:** Automating repetitive tasks, such as processing each item in a list.

While / else

```
# ------------------------------------------------------------
# while / else — Count to 5 with Else Clause
# ------------------------------------------------------------
# A 'while' loop repeats its body as long as the condition
# (count <= 5) remains True. We MUST update 'count' inside
# the loop; forgetting to do so creates an infinite loop.
#
# The 'else' block runs after the while condition becomes
# False (i.e. the loop ended naturally, not via 'break').
# At that point count = 6, which is printed by else.
# Output: 0 1 2 3 4 5 6
# ------------------------------------------------------------
count = 0
while count <= 5:          # loop continues while count is 0-5
    print(count)
    count = count + 1      # increment; prevents infinite loop
else:
    print(count)           # count is now 6 (condition just failed)
```

Trace the output without running the code.

How to early exit from a while loop?

In **Week2.ipynb** , check the section **“Loops in Programming”**

In **Week2_Exercise.ipynb** , you have 30 minutes to solve “Section B”.

Pause / Break

See you in 15 minutes ☺!
Please be on time..

Basic List Comprehensions

Definition

A **list comprehension** is a concise way to create a list by combining:

- iteration
- optional conditional filtering
- an output expression

It replaces longer `for` loop constructions with a single, readable line.

General Syntax

```
[expression for item in iterable if condition]
```

Components

- `expression` → the value added to the new list
- `item` → the variable representing each element
- `iterable` → the sequence being iterated over
- `if condition` → optional filter to include only certain elements

Computational Insight

A list comprehension:

```
[expression for item in iterable if condition]
```

is conceptually equivalent to:

```
for item in iterable:
    if condition:
        result.append(expression)
```

but executed in a more compact and often more efficient manner.

Basic List Comprehension - Squaring Numbers

```python
# ------------------------------------------------------------
# List Comprehension – Squaring Numbers
# ------------------------------------------------------------
# [x * x for x in range(5)] builds a new list by computing
# x squared for every x in {0, 1, 2, 3, 4}.
# The standard loop equivalent is shown in the docstring.
# Note: x**2 and x*x are equivalent; both express squaring.
# ------------------------------------------------------------
squares = [x * x for x in range(5)]   # expression: x*x, iterable: range(5)
print(squares)

"""
Standard Loop equivalent:
squares = []
for x in range(5):
    squares.append(x * x)
"""
```

```
[0, 1, 4, 9, 16]
```

In **Week2.ipynb** , check the section **“ Basic List Comprehensions as Compact Logic Expressions ”**

In **Week2_Exercise.ipynb** , you have 35 minutes to solve “Section C”.

Final Mini Challenge: Smart Student Performance Analyser

Final Mini Challenge: Smart Student Performance Analyser

You are given a dataset of students, where each student has a name, a score, and an attendance percentage.

Given Data

```
students = [
    {"name": "Ali", "score": 85, "attendance": 92},
    {"name": "Sara", "score": 58, "attendance": 75},
    {"name": "Lina", "score": 73, "attendance": 88},
    {"name": "Omar", "score": 49, "attendance": 95},
    {"name": "Noor", "score": 91, "attendance": 98},
    {"name": "Yara", "score": 66, "attendance": 60}
]
```



In **Week2_Exercise.ipynb** , you have 25 minutes to solve the “Final Mini Challenge”

What did we learn today?

Logical Reasoning in Programming

Conditional statements, decision structures, and shorthand conditional (Ternary operator)

Loops in Programming

For Loop, Nested Loops, Loop Control statements, and While Loop

Basic List Comprehensions

Compact Logic Expressions

Coming Up : Week (3)

So far, we have worked primarily with individual variables and basic control flow. How would you design a system that stores and organises multiple related pieces of information?

• Lists and indexing principles • Tuples and immutability
• Dictionaries and key–value modelling • Sets and uniqueness
• Nested data structures for real-world data modelling

Any Questions?

See you next time ☺

Dr. Ghadeer Mobasher
ghadeer.mobasher@srh-hochschulen.de
https://www.linkedin.com/in/ghadeermobasher/
LGS 6, 2. OG, Arc 213